Week 1: Hashing and two pointers

Nearby duplicates in sensor logs

Retransmission produces the event log `[4, 7, 4, 4]`. We need to detect equal values at most one position apart. Comparing only the first and third 4 misses the final pair.

Repeatedly rescanning history becomes expensive as the log grows. This week retains only the past information needed by the question and uses sorted order to eliminate impossible candidates.

Week 1 problems and learning outcomes

Session 1 compares array, set, and dictionary costs, then links values to earlier positions. Session 2 justifies pointer moves in sorted values and extends the idea to three-value sums.

- Define inputs, outputs, and constraints first.
- Explain the stored state and why discarded candidates cannot be answers.
- Separate time, auxiliary storage, and output costs.

Practice uses LeetCode 1, 219, 167, and 15.

Information in lists, sets, and dictionaries

A list preserves positions and duplicates. A set stores membership. A dictionary associates one value with each key. The same log calls for different stored information depending on the question.

```
events = [4, 7, 4, 4]
seen = {4, 7}
latest = {4: 3, 7: 1}

print(events[2])        # 4
print(4 in seen)        # True
print(latest[4])        # 3
```

Input size and operation costs

n is the number of records. These bounds use fixed-width integers and Python array/hash containers.

Operation	Cost	Reason
List indexing	O(1)	Direct position access
List value search	O(n)	Sequential comparisons
dict/set lookup/update	Expected O(1)	Key lookup; worst O(n)
Slice/copy q entries	O(q) time/space	Creates new references
Sort n entries	Worst O(n log n)	Python workspace O(n)

Big-O gives an upper bound calculated for the worst case as input size grows.

Hash keys and object copies

Integers, strings, and tuples containing only hashable elements can be keys. A mutable list cannot. `dict.values()` is a view; `list(...)` creates a list.

```
latest = {4: 2}
snapshot = latest.copy()
latest[4] = 3
print(snapshot[4], latest[4])  # 2 3

key = (2, 0, 1)
counts = {key: 5}
print(counts[key])            # 5
```

Account separately for hashing long strings or tuples. Integer-key hashing is treated as constant here.

LeetCode 1: contract and baseline

Two Sum: return indices at distinct positions summing to target. The source guarantees one answer; one position cannot be reused.

For `nums=[4,8,3], target=7`, return `[0,2]`. Enumerating every pair needs no stored history.

```
def two_sum_brute(nums, target):
    for left in range(len(nums)):
        for right in range(left + 1, len(nums)):
            if nums[left] + nums[right] == target:
                return [left, right]
    return []
```

Pair counts and repeated searches

There are this many distinct index pairs. For n=10,000, the count is 49,995,000. An answer may be found earlier, but the worst case examines a quadratic number of candidates.

$$\frac{n\,(\,n-1\,)}{2}$$

If the current value is 3 and target is 7, the needed earlier value is 4. Store its presence and position instead of comparing every earlier value again.

Earlier-value state for Two Sum

`seen[value]` records an earlier position. Looking up before recording the current position ensures that the two returned indices differ.

```
def two_sum(nums, target):
    seen = {}
    for index, value in enumerate(nums):
        if target - value in seen:
            return [seen[target - value], index]
        seen[value] = index
    return []  # Defensive extension: official inputs have one answer.
```

Tracing Two Sum

index	value	Needed	seen before lookup	Result
0	4	3	{}	Absent → record 4:0
1	8	-1	{4:0}	Absent → record 8:1
2	3	4	{4:0,8:1}	Found → [0,2]

The map contains the distinct values already read, never future positions.

Two Sum correctness and update order

At loop entry, seen contains only earlier positions. When the later position of the answer is reached, the earlier partner is present.

For `nums=[3,3], target=6`, recording first can make lookup return the current index itself. The first 3 should fail lookup and be stored; the second then returns `[0,1]`.

One scan gives expected O(n) time and O(n) auxiliary storage. General worst-case hash collisions give an $O(n^2)$ upper bound.

LeetCode 219: adding a distance condition

Contains Duplicate II: decide whether equal values at distinct positions are at most k apart.

`nums=[4,7,4,4], k=1` is True because of the final two entries. With k=0, no distinct indices qualify.

Rescanning up to k earlier positions per entry costs O(n·min(n,k)); slicing also copies each interval. Which single past occurrence per value is sufficient?

Meaning of the latest index

`latest[value]` is the most recent occurrence. It is the nearest past occurrence, so if it is too far, all older ones fail.

index	value	Prior latest[value]	Distance	Action
0	4	Absent	-	Store 4:0
1	7	Absent	-	Store 7:1
2	4	0	2 > 1	Fail, update to 4:2
3	4	2	1 <= 1	Return True

Implementing latest-index state

```
def nearby_duplicate(nums, k):
    latest = {}
    for index, value in enumerate(nums):
        if value in latest and index - latest[value] <= k:
            return True
        latest[value] = index
    return False
```

Update the latest position even when a repeated value fails the distance check. Keeping only the first occurrence misses the final pair in the example.

Correctness and storage of recent state

At current position j, the largest earlier matching index i minimizes j-i. If it is within k, an answer exists; otherwise every older occurrence fails. Updating preserves this meaning for the next iteration.

n is length and d the number of distinct values. Expected time is O(n), auxiliary storage O(d), with d<=n. This implementation retains old keys.

Check: explain why all previous indices of value 4 need not be stored.

Session 1 concept checks

- What do `4 in events` and `4 in latest` each test?
- When does recording before lookup go wrong on `[3,3]`?
- Which answer is missed if index 2 does not update state in `[4,7,4,4], k=1`?
- How does copying latest on every iteration change complexity?

Write answers before opening the instructor checks.

Pairing sorted part lengths

Find two part lengths summing to 7 in sorted `[1,4,6,9]`. The endpoints 1 and 9 total 10. Why is it safe to discard 9?

Session 2 uses order to eliminate many candidates at once. Before memorizing pointer moves, identify every pair removed by each move.

The input contract of LeetCode 167

Two Sum II - Input Array Is Sorted: input is nondecreasing and has one answer. Use distinct elements, return one-based positions, and use constant extra space.

```
numbers=[1,4,6,9], target=7 → [1,3]
```

This problem requires O(1) auxiliary space. Storing earlier values in a dictionary uses up to O(n) space, so we apply two pointers to the sorted input.

Eliminating a left endpoint with a small sum

The candidate interval is `[left,right]`. If its endpoint sum is below target, every available partner for left is at most the right value. All those sums are too small.

Incrementing left removes only failed pairs using that endpoint. Decreasing right cannot repair a sum that is too small.

Invariant: if an answer has not yet been found, both its positions remain inside the interval.

Eliminating a right endpoint with a large sum

If the endpoint sum exceeds target, every available partner for right is at least the left value. All those sums are too large.

Current pair	Sum	Decision for target=7	Removed candidates
1,9	10	Too large	9 paired with 1,4,6
1,6	7	Answer	No further elimination

The condition is left<right because one element cannot be used twice.

Implementing two pointers

```
def two_sum_sorted(numbers, target):
    left, right = 0, len(numbers) - 1
    while left < right:
        total = numbers[left] + numbers[right]
        if total == target:
            return [left + 1, right + 1]
        if total < target:
            left += 1
        else:
            right -= 1
    return []
```

Use zero-based indices inside the loop; add one only when returning.

Termination and total pointer movement

Each iteration returns an answer or moves one pointer inward. The gap shrinks, ensuring termination.

left moves only rightward and right only leftward. Their total movements are linear, giving O(n) time and O(1) auxiliary space.

Outside the sorted contract, `[3,1,4], target=5` makes the rule miss 1 and 4. The elimination depends on the source problem’s sorting guarantee.

LeetCode 15: three values and duplicate triplets

3Sum: return unique value triplets summing to zero at distinct positions. Equal values remain usable when their positions differ.

`[-2,0,1,1,2] → [[-2,0,2],[-2,1,1]]`

Enumerating three positions costs $O(n^3)$. Fixing one value leaves a two-value target equal to its negative.

Sorting and the fixed value

Sort a copy, fix index as the first position, and set left=index+1 and right=n-1. The three positions are then distinct.

Move left for a negative three-value sum and right for a positive one. This uses the same sorted elimination as 167.

3Sum returns value combinations, so we search the sorted array. Problems that return original indices require storing each value together with its original index.

Tracing 3Sum with fixed value -2

index,left,right	Values	Sum	Action
0,1,4	-2,0,2	0	Store [-2,0,2]; move both
0,2,3	-2,1,1	0	Store [-2,1,1]; move both
0,3,2	Interval ended	-	Finish fixed value -2

The next fixed value is 0. No pair from 1,1,2 sums to its negation, zero. Separate the search for one fixed value from subsequent fixed values.

3Sum implementation: fixing and moving

This page and the next contain consecutive portions of one function.

```
def three_sum(nums):
    values = sorted(nums)
    result = []
    for index in range(len(values) - 2):
        if index and values[index] == values[index - 1]:
            continue
        left, right = index + 1, len(values) - 1
        while left < right:
            total = values[index] + values[left] + values[right]
            if total < 0:
                left += 1
```

3Sum implementation: recording and duplicate skipping

```
            elif total > 0:
                right -= 1
            else:
                result.append([values[index], values[left], values[right]])
                left += 1
                right -= 1
                while left < right and values[left] == values[left - 1]:
                    left += 1
                while left < right and values[right] == values[right + 1]:
                    right -= 1
    return result
```

After storing a match, move both pointers and skip values equal to the ones just used.

Two places to remove duplicate output

First, skip a fixed value equal to the previous fixed value. Second, after a match skip repeated left and right values. Equal values at distinct positions can be used together. Duplicate removal targets value combinations already returned.

For `[0,0,0,0]`, several index choices produce the single value triplet `[0,0,0]`. In `[-2,0,1,1,2]`, `[-2,1,1]` is valid because the two 1s occupy different positions.

3Sum correctness and cost

Every sorted triplet has a smallest value. When it is fixed, the two-pointer search includes its remaining pair and eliminates only impossible sums. Duplicate skipping removes repeated representations of the same value triplet.

n is input length and z the output count. Sorting a copy costs O(n log n); each fixed value takes O(n) pointer movement, giving $O(n^2)$ total time. Copy/sort workspace is O(n), and stored output is O(z).

Choosing state from the contract

Question	Retained state	Contract to check
Unsorted pair sum	Earlier value→position	No index reuse
Nearby equal values	Value→latest position	Distance k; update after failure
Sorted pair sum	left,right	Sorting, constant space, one-based output
Unique triplet sums	Fixed index and two pointers	Distinct indices, unique value triplets

Identify the information needed for the next decision and candidates that can be discarded before naming a data structure.

Independent exercises and submission

Solve LeetCode 1, 219, 167, and 15 in the student exercises. Implement student_week_01.py and trace a small input by hand first.

Submit state meaning, correctness, complexity, and boundary inputs with each solution. Use the staged hints and solutions only as far as needed.

Run `python week01_walkthrough.py` from the Python bundle to check the lecture examples and traces.

Core sources

Erik Demaine and Srini Devadas, Introduction to Algorithms, MIT 6.006 (Fall 2011).

Use each official LeetCode problem link for its complete input contract. Next week extends state updates to contiguous windows and unresolved information in stacks.